

Leveraging GitHub Social Network for Enhanced Link Prediction and Node Classification

SeungHyun Nam and Jiaqi Yu

AS&E, University of Rochester

Abstract. This paper explores the application of machine learning algorithms on a GitHub social network dataset from the UCI Laboratory to enhance link prediction and node classification capabilities. The dataset comprises 37,700 GitHub users and over 4,000 personal attributes, making it a fertile ground for examining social interactions and connectivity. Through empirical analysis, we evaluate several algorithms, discovering that simpler methods occasionally outperform more complex ones depending on specific conditions. Our results highlight the effectiveness of models like Node2Vec and Graph Convolutional Networks (GCN) in predicting potential links and classifying nodes, thereby facilitating more accurate and personalized user interactions on social platforms. The study aims to refine these predictive algorithms to improve social networking services, thereby aiding in community building and enhancing user engagement. We conclude with insights into algorithmic performance through a detailed evaluation matrix comprising accuracy, precision, recall, and F1-score, providing a robust framework for future advancements in social network analysis.

Keywords: Link Prediction · Node Classification · Node2Vec · GCN · Social Network Analysis.

1 Introduction

1.1 Problem Statement

Nowadays, almost every single person with a phone uses several social networks. With this trend going on, we are expecting more and more young people to start learning to use social media in the following decades. With so many new users coming in, it would be very important for social networks to provide specific and accurate node classifications. These recommendations should satisfy new users when their features are noticed by the platform or identified by themselves. What's more, social network analysts may also want to know possible communities within social networks, so it's also crucial to apply link prediction to community detection.

After considering this situation, we found a dataset from UCI Laboratory that focuses on GitHub users. The dataset contains 37,700 instances from GitHub, whom each has connection with, and more than 4,000 personal features.

We performed several machine learning algorithms on the dataset we acquired in order to achieve high-quality link prediction. After analyzing each algorithm, we came to a conclusion that easier approaches like random forest and Node2Vec performed better on our dataset.

For outcome we used a matrix of accuracy, precision, recall, and F1-score. We compared all four values in order to evaluate an algorithm.

The project aims to develop robust algorithms and machine learning models that can be applied to this and potentially other datasets for high-efficiency and accurate prediction tasks. By enhancing link prediction and node classification, we strive to improve collaboration and community building within GitHub, addressing current inefficiencies.

1.2 Literature Review

Let's first identify the definition of link prediction. According to Nur Nasuha Daud's review[2], Link prediction is a study that assumes the upcoming behaviors in social networks, and link prediction methods anticipate the likelihood of a future connection between two nodes in a given network.

Detection, recommendation, and influence analysis are a few applications of link prediction.

Anomaly detection, community detection, and event detection are very important uses of link prediction, and we have tried event detection ourselves while doing lab 1. As for recommendation, a recommendation system provides users with a more favorable social environment, which has more related information and more information they might want to see. When it comes to influence analysis, it would be interesting if we could quantify the strength of social influences by friends or family members[8].

Here we address a few algorithms researchers have proposed in recent years:

Tang proposed Topical Affinity Propagation to model the topic-level social influence on large networks[7].

Mohan proposed an advanced community detection system that utilized link prediction algorithms and a Bulk Synchronous Parallel programming model[5].

Dong Used recommendations across different heterogeneous networks to achieve friend recommendations and evaluated their result by using the ROC curves and AUC measures[3].

2 Test Data

2.1 Data Acquisition

The dataset we used is the GitHub Social Network, characterized by its non-directed nature, binary-labeled node features, and absence of temporal and edge features.

We downloaded the dataset from the uci repository, and here's the **link** for it.

With 37,700 nodes and 289,003 edges, the undirected dataset offers a rich ground for exploring link prediction, friend recommendation, community detection, and network visualization tasks.

2.2 Data EDA

Visually Related Analysis: At first we tried to use the full graph to create an image. In figure 1, we used Jaccard coefficients as a node's weight and it turned out to be a mess due to the huge number of nodes and relations we have in the graph.

Since it's very hard to tell anything from figure 1, instead of using all data, a list of random 500 nodes in the network is compiled, and this list is used to create a subgraph that only includes these nodes in figure 2.

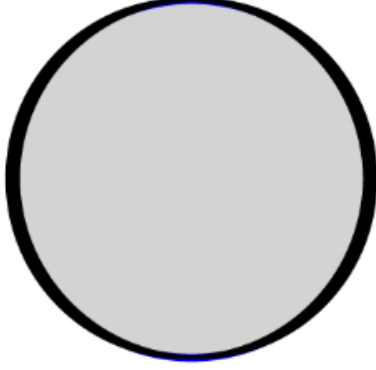


Fig. 1. The whole dataset visualized with all edges weighted by Jaccard Coefficients

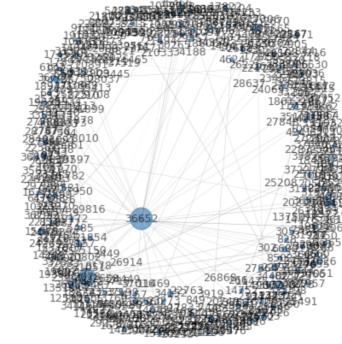


Fig. 2. Circular Layout with Node Centrality and Edge Weights

Here is the equation of Jaccard Coefficient value:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

From figure 2 we could tell our data is a highly connected graph, and high degree nodes and low degree nodes both exist in the graph.

Data Based Analysis:

- **Degree Centrality:** By building a graph with Networkit[6], we found the highest degree in our graph is 9458, and the lowest is 1. An average degree is 15.33. This means the degree distribution is highly skewed, with some famous engineers having a lot of followers. Here in figure 3 we can see how the degree of nodes distribute clearly.

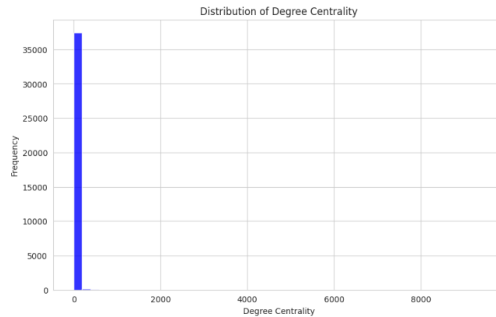


Fig. 3. Histogram of Degree Centrality Distribution

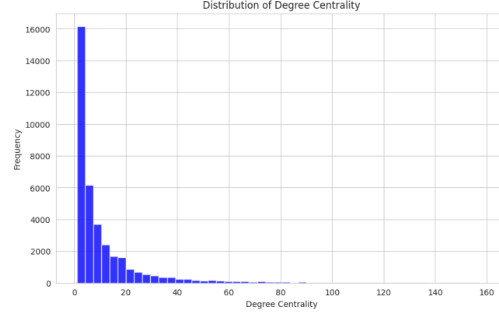


Fig. 4. Histogram of Degree Centrality Without Greater than 160 Degrees

- **Degree Centrality, With Nodes Removed:** To get a better look, we removed every single node with degrees greater than 160, and got figure 4. The distribution in figure 4 is still highly skewed, with most nodes having only 1 or 2 degrees. However, here we noticed a power law distribution in the figure.
- **Power Law:** Fitting our dataset to `powerlaw.fit()` gives us an α of 2.573, and a x_{min} of 58.0. The power law graph is shown in figure 5.

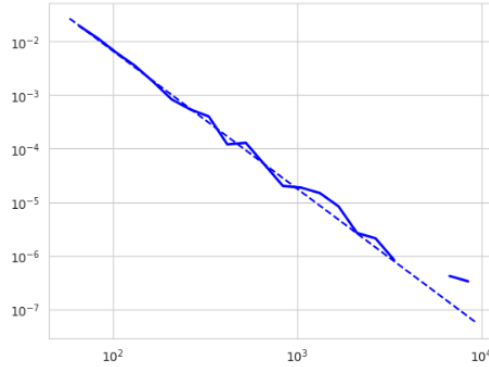


Fig. 5. Power Law Fit

A small α that is between 2 and 3 means our data does not contain a heavy tail, and a x_{min} of 58.0 indicates only nodes with degrees greater than 58 follows power law distribution. Since nodes with degrees greater than 58 only consists of a very small amount of nodes, removing this part of people will significantly disrupt the network. These nodes are also classified as key nodes in our graph.

- **Log-Log Plot:** Figure 6 is a log-log plot illustrating our network's degree distribution, showcasing a linear trend typical of power-law distributions. The linearity observed, particularly in the middle range, confirms our network as scale-free, characterized by a few highly connected hubs. Most nodes are lightly connected, contributing to a long tail in the degree distribution that highlights network heterogeneity.

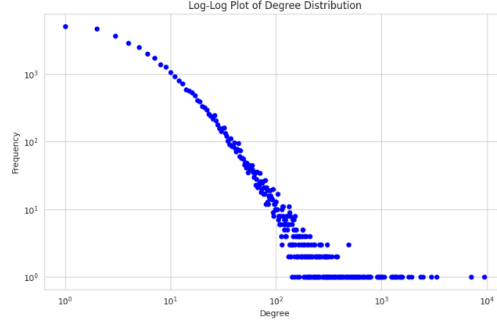


Fig. 6. Log-Log Plot

Such a pattern indicates general robustness against random node failures but reveals a vulnerability to targeted attacks on these central hubs, which could disrupt network operations significantly.

Additionally, some scatter in the higher degree values suggests slight deviations from an ideal power-law, possibly reflecting the complex and irregular nature of our network.

- **Assortativity:** Our data has an assortativity of -0.075, which is a negative number very close to zero. This indicates each node has a slight tendency in connecting to others with dissimilar degrees.

This is a typical shape of a social network, in which few leaders are connected to many subordinates, but not as much to each other.

- **Eccentricity:** Eccentricity is the furthest steps a node can be from others. Figure 7 is a histogram of the eccentricities' distribution of our data.

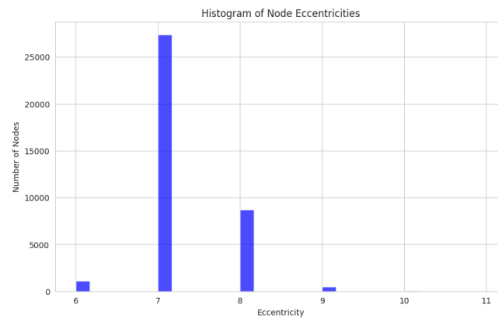


Fig. 7. Histogram of Eccentricities' Distribution

This figure is unimodal, with a peak at 7. It shows a rapid decline on both sides of this peak. The eccentricities range from 6 to 11, which suggests that the network might not be highly compact.

3 Technology Roadmap

3.1 Methodology and Models

In our study, we systematically applied three distinct methods to both node classification and link prediction tasks within network analysis. These methods range from fundamental statistical models to advanced machine learning techniques, each progressively building on the insights and technologies of the previous. Our approach ensures a comprehensive exploration of the capabilities and potential improvements in predicting both node attributes and link formations within the graph structure. This methodology section details the sequential application of these methods, beginning with our basic models in node classification.

Node Classification

- **Basic Models (Method one):** Initially, we employed both RandomForest and Logistic Regression. RandomForest, a decision tree-based ensemble method, adeptly handles non-linear data. Logistic regression, which provides a probabilistic approach, is effective for binary classification. These models establish a baseline performance for comparison.
- **Enhanced Feature Engineering (Method two):** We improved our model’s capabilities by integrating techniques such as One-hot Encoding and Node2Vec. One-hot Encoding efficiently manages categorical data, while Node2Vec leverages node embeddings to preserve neighborhood information in the network, enhancing feature representations for both Random Forest and Logistic Regression.
- **Advanced Methods (Method three):** We explored using Graph Convolutional Networks (GCN), which represent a significant advancement in processing graph data. GCN excel in capturing deep relational information embedded within the graph, crucial for tasks where node relationships significantly impact outcomes.

Link Prediction

- **Basic Models (Method one)** We initiate our exploration with RandomForest and Logistic Regression, foundational models in statistical and machine learning tasks. RandomForest, with its ensemble of decision trees, offers a diverse perspective by synthesizing multiple models, enhancing robustness and reducing overfitting. Logistic Regression, conversely, applies a probabilistic framework ideal for binary classifications, such as predicting the presence or absence of a link between nodes.
- **Enhanced Feature Engineering (Method two)** In our second approach, we refine our model’s inputs using One-hot Encoding and Node2Vec. One-hot Encoding transforms categorical attributes into binary vectors, essential for Logistic Regression. Node2Vec, on the other hand, generates embeddings that encapsulate the neighborhood topology, significantly improving the predictive capabilities of our models. These techniques allow both RandomForest and Logistic Regression to better harness the structured feature space for enhanced performance.
- **Advanced Methods (Method three)** Finally, we employ Graph Convolutional Networks (GCN)[9]. GCN represent a significant advancement by integrating graph topology directly into the learning process. This method leverages the rich relational information embedded within the graph, which is crucial for accurately predicting potential links in network structures, thus profoundly impacting our predictions.

3.2 Algorithms

The algorithms we used to perform link prediction are **Graph Convolutional Networks (GCN)** and **Node2Vec**[4]. They represent two distinct approaches for learning representations on graph-structured data.

Node2Vec: Node2Vec adopts a flexible strategy by generating embeddings through random walks on the graph. This method captures the local topology of nodes in a more scalable manner, as it simplifies the graph structure into sequences that resemble sentences in natural language processing, which are then processed using techniques similar to word embeddings methods like Word2Vec.

Node2Vec is particularly effective at preserving the community structure and can be easily applied to any graph without the need for significant computational resources.

However, it might not capture the hierarchical structure of the graph as effectively as GCN, and its performance can be sensitive to the choice of hyper-parameters for the random walks.

Algorithm 1 Node2Vec Algorithm Pseudocode

Require: Graph $G(V, E)$, Dimension dim , Walk length $walk_length$, Number of walks num_walks , Workers $workers$

- 1: Initialize Node2Vec model $Node2Vec(G, dim, walk_length, num_walks, workers)$
- 2: Train model with parameters: window size 10, min count 1, batch words 4
- 3: Extract node embeddings from the model
- 4: **for** each node id in G **do**
- 5: Assume one-hot encoding for node features
- 6: **end for**
- 7: Load node labels
- 8: Split the data into training and testing sets
- 9: Train a RandomForestClassifier
- 10: Predict on the test set
- 11: Calculate and print accuracy, precision, recall, and F1 score

Ensure: Node embeddings, Model performance metrics

Model Architecture: Node2Vec employs a flexible architecture that is not tied to the graph structure, unlike GCNs. It uses a series of random walks to efficiently explore diverse neighborhoods. This stochastic process results in a rich embedding that captures both the local and global context of nodes, which is crucial for effective machine learning predictions.

Training and Optimization: The training of Node2Vec involves setting the dimensionality of the embeddings and the length and number of random walks to balance between the breadth and depth of neighborhood exploration. The use of soft constraints like window size, minimum count, and batch words during training helps refine the quality of the embeddings. Optimization is achieved through stochastic gradient descent, ensuring the embeddings are well-suited for downstream tasks such as classification.

Forward Pass and Model Training: During the forward pass, Node2Vec processes each node independently, allowing parallel processing and significantly speeding up the embedding generation. The training phase involves using the embeddings as features in traditional classifiers like RandomForest, leveraging the rich, structured information embedded in the vectors to effectively predict node labels or link formation.

GCN: GCN operate by leveraging the graph structure directly, employing recursive neighborhood aggregation or message passing mechanisms to embed nodes in a way that captures both their features and the topology of the graph. This approach allows GCN to generate embeddings that are inherently structured by the graph, making them powerful for tasks where the graph structure is crucial, such as node classification, link prediction, and graph classification.

However, GCN can be computationally intensive, especially for large graphs, due to the recursive aggregation process. They also require careful design to avoid over-smoothing, where node embeddings become indistinguishable after several layers of aggregation.

Algorithm 2 Graph Convolutional Network Training Procedure

```

1: Input: Graph  $G(V, E)$  with node features  $X$ , edge list  $E$ 
2: Parameters: Weight matrices  $W^{(1)}, W^{(2)}, W^{(3)}$ , learning rate  $\eta$ 
3: procedure GCNFORWARD( $X, E$ )
4:    $H^{(0)} \leftarrow X$ 
5:   for  $l = 1$  to 3 do
6:      $H^{(l)} \leftarrow \text{ReLU}(\text{GCNConv}(H^{(l-1)}, E, W^{(l)}))$ 
7:      $H^{(l)} \leftarrow \text{Dropout}(H^{(l)}, 0.5)$ 
8:   end for
9:   return  $H^{(3)}$ 
10: end procedure
11: procedure TRAINGCN( $G, X, E$ )
12:   Initialize parameters  $W^{(1)}, W^{(2)}, W^{(3)}$ 
13:   for each epoch do
14:      $Z \leftarrow \text{GCNFORWARD}(X, E)$ 
15:     Compute loss  $\mathcal{L}$  using BCEWithLogitsLoss
16:     Backpropagate to compute gradients
17:     Update weights using Adam optimizer
18:   end for
19: end procedure

```

The Graph Convolutional Network (GCN) developed for this project is structured to optimize the processing of graph-based data, specifically targeting the efficient and effective analysis of node relationships within a network. The architecture comprises three convolutional layers, each tailored to capture and integrate the features of node neighborhoods to facilitate accurate predictions.

Model Architecture: Our GCN model features an input layer that handles 42 dynamic features per node, ensuring a comprehensive representation of each node's attributes. The first and second layers of the network are equipped with 32 hidden units each, designed to amplify the signal from the node features through nonlinear transformations.

The third layer, containing 16 hidden units, focuses on refining the feature representations to yield a more precise output. Each convolutional layer is followed by a ReLU activation function to introduce non-linearity and a dropout mechanism with a dropout rate of 0.5 to prevent overfitting and promote generalization across unseen data.

Training and Optimization: Training the model involves the use of an Adam optimizer with an initial learning rate of 0.01, which is adjusted downward each epoch by an exponential decay rate of 0.995. This approach helps in fine-tuning the learning process over time, making the model adaptive to changes in the data dynamics. The objective function used is the Binary Cross-Entropy with Logits Loss, ideal for the binary classification tasks inherent in edge prediction scenarios within the network.

Forward Pass and Model Training: During the forward pass, the model employs negative sampling to create a balanced learning environment by generating negative edge samples. This is crucial for training robust models capable of distinguishing between existing and non-existing links. The model computes scores for both positive and negative edges by summing the product of connected node embeddings, which are then used to compute the loss for backpropagation. Training is iterative and includes monitoring of the loss to implement an early stopping mechanism. If the loss does not improve over 1000 epochs, training is halted, and the best model state is saved, providing a checkpoint for potentially optimal performance.

In summary, GCN offer a powerful but computationally demanding approach that is deeply integrated with the graph structure, while Node2Vec provides a more scalable and flexible method that excels at capturing community-based similarities.

4 Results and Discussion

4.1 Results

Here we present a comprehensive analysis of the results obtained from our experimental models in node classification and link prediction tasks. We calculated the performance metrics containing accuracy, precision, recall, and F1 score to measure the effectiveness of different machine learning models under various feature engineering techniques, namely Node2Vec (N2V) and One-hot Encoding (OHE), as well as their absence.

Node Classification: Table 1 presents the metrics for our node classification models. The performance of models incorporating Node2Vec (N2V) and One-hot Encoding (OHE) was robust. Logistic Regression, for instance, achieved an accuracy of 0.8573 and an F1 score of 0.7001, underlining the efficacy of embedding-based features. Surprisingly, the exclusion of N2V and OHE led to a slight improvement in the accuracy and F1 score for Logistic Regression, suggesting that in some scenarios, a simpler model configuration may suffice, depending on data characteristics.

However, the Three Layer Graph Convolutional Network (GCN) model, while innovative, did not perform as well as the more traditional models. It reached an accuracy of only 0.7369 and an F1 score of 0.7360, indicating that for our specific dataset, traditional feature engineering might still hold an edge over raw graph-based learning approaches.

Table 1. Node Classification Metrics

Model	Accuracy	Precision	Recall	F1 Score
With N2V, OHE				
Random Forest	0.8544	0.8064	0.5877	0.6799
Logistic Regression	0.8573	0.7830	0.6331	0.7001
Without N2V, OHE				
Random Forest	0.8569	0.7995	0.6089	0.6913
Logistic Regression	0.8576	0.7830	0.6346	0.7010
GNN				
Three Layer GCN	0.7369	0.6987	0.7252	0.7360

Link Prediction: Turning to link prediction, Table 2 highlights significant differences in model performance. The Random Forest model, when enhanced with Node2Vec and One-hot Encoding, stood out, securing an F1 score of 0.9341. This illustrates the capacity of embedding-based features to capture complex node interactions within the model framework effectively.

Conversely, Logistic Regression showed less effectiveness in the same enhanced settings, with an F1 score of only 0.7200, implying that simpler linear models may struggle to handle the complexity of embedded spaces.

Similar to node classification, GCN did not perform as well in link prediction. This reiterates the need for potentially more advanced graph neural network architectures or better tuning to utilize their theoretical benefits fully.

Table 2. Link Prediction Metrics

Model	Accuracy	Precision	Recall	F1 Score
With N2V, OHE				
Random Forest	0.9362	0.9649	0.9053	0.9341
Logistic Regression	0.7121	0.7004	0.7407	0.7200
Without N2V, OHE				
Random Forest	0.8479	0.9429	0.7393	0.8288
Logistic Regression	0.8445	0.9515	0.7246	0.8227
GNN				
Three Layer GCN	0.7063	0.6801	0.7789	0.7262

4.2 Interpretation

The node classification results underscore the RandomForest model’s high accuracy and precision, indicating its effectiveness in classifying nodes based on their features. The lower recall metric signifies a need for refinement, suggesting the model’s tendency to overlook certain positive classifications.

Conversely, the link prediction outcomes reveal the logistic regression model’s aptitude in fitting the binary classification nature of the task, with a commendable ability to generalize from training to unseen data. Nevertheless, there exists potential for enhancing the model’s discriminative power, as suggested by the ROC AUC metric.

5 Difficulties and Unsolved Problems

As we advance our research and apply novel computational methods to complex graph-based problems, we have encountered several significant challenges. These challenges not only test the limits of our existing methodologies but also highlight the intricate complexities involved in modeling and predicting using advanced techniques such as Generative Adversarial Networks (GANs)[1] and Graph Convolutional Networks (GCNs).

5.1 Challenges in GANs

As we progress with implementing cutting-edge technologies, we have encountered some challenges that push the boundaries of our understanding and capabilities. In our case, the use of GANs for link prediction.

Firstly, the GAN model's initial performance was lower than expected. This was especially evident in tasks like link prediction and node classification, where accuracy was significantly lower in initial tests.

We made numerous adjustments to parameters such as learning rates, batch sizes, activation functions, and dropout layers. Despite these modifications, only marginal improvements in model accuracy were observed after increasing the number of training epochs.

This persistent low improvement underscores a deeper issue—perhaps indicating that these problems cannot be solved by parameter tuning alone, and may require a more fundamental reevaluation of the model's design or the approach to the data.

5.2 Challenges with GCN Implementation

Our experiences with the Graph Convolutional Network (GCN) for node classification and link prediction have also highlighted several key issues.

Initially, the performance of our GCN model was not up to the anticipated standards. This was particularly noticeable in the context of link prediction, where the model's accuracy and loss metrics did not meet our expectations from the outset.

We undertook extensive parameter tuning, modifying learning rates, the number of layers, dropout rates, and other hyper-parameters in an effort to enhance the model's effectiveness. Despite these efforts, the improvements in performance were minimal, suggesting that the issues might be more intrinsic to the model's architecture or our approach to the graph data itself.

This ongoing struggle with optimizing the GCN underscores the need for possibly rethinking the model framework or exploring alternative methodologies that might better capture the complexities of graph-based data.

6 Future Works

Looking ahead, there are several aspects we plan to explore to enhance the effectiveness of our models, both GANs and GCN.

6.1 GAN model improvement

For the GAN models, we intend to further optimize the parameters and explore more advanced techniques within the GAN framework.

We believe that extending the training duration and increasing the complexity of the training datasets might yield better learning outcomes.

Additionally, collaborative research with other initiatives could provide fresh insights that lead to breakthroughs in model effectiveness.

6.2 GCN model improvement

For the GCN models, our improvement strategy is to conduct more detailed investigations into hyperparameter settings and assess new architectural configurations to improve the model's ability to process and analyze data.

We intend to enrich the training process by incorporating a wider variety of datasets, which could help refine the model's performance.

Also, we may be able to advance methods that will enhance the model's transparency and fairness, guaranteeing that our outcomes are both precise and unbiased.

As we conclude this exploration of leveraging GitHub's social network for enhanced link prediction and node classification, we reflect on the significant strides we've made as well as the challenges that remain. Through rigorous empirical analysis and the application of both traditional and advanced machine learning techniques, our study has not only expanded our understanding of the intricacies of social network dynamics but also underscored the potential for further advancements. As we move forward, the insights garnered from this research will undoubtedly serve as a valuable foundation for future work aimed at refining predictive algorithms and improving social networking services, thereby fostering more meaningful interactions and robust community networks.

References

1. Jesús Bobadilla and Abraham Gutiérrez. Wasserstein gan-based architecture to generate collaborative filtering synthetic datasets. *Applied Intelligence*, 2024:1–19, 2024.
2. Nur Nasuha Daud, Siti Hafizah Ab Hamid, Muntadher Saadoon, Firdaus Sahran, and Nor Badrul Anuar. Applications of link prediction in social networks: A review. *Journal of Network and Computer Applications*, 166:102716, 2020.
3. Yuxiao Dong, Jie Tang, Sen Wu, Jilei Tian, Nitesh V Chawla, Jinghai Rao, and Huanhuan Cao. Link prediction and recommendation across heterogeneous social networks. In *2012 IEEE 12th International conference on data mining*, pages 181–190. IEEE, 2012.
4. Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
5. Anuraj Mohan, R Venkatesan, and KV Pramod. A scalable method for link prediction in large real world networks. *Journal of Parallel and Distributed Computing*, 109:89–101, 2017.
6. Christian Staudt, Aleksejs Sazonovs, and Henning Meyerhenke. Networkit: An interactive tool suite for high-performance network analysis. *arXiv preprint arXiv:1403.3005*, 2014.
7. Jie Tang, Jimeng Sun, Chi Wang, and Zi Yang. Social influence analysis in large-scale networks. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 807–816, 2009.
8. Stanley Wasserman and Katherine Faust. Social network analysis: Methods and applications. 1994.
9. Si Zhang et al. Graph convolutional networks: a comprehensive review. *Computational Social Networks*, 6(1):1–23, 2019.